
Coursework

Thomas Bending

2025-26

This coursework is worth 75% of the module. You should submit your work online via the module's site on My Learning by the end of **Friday 10 April 2026**. You are strongly advised to submit your work well before the deadline in case of possible network issues.

Tasks

In the module's first session we looked at the problem of finding the shortest route from one city to another, given the distances between various pairs of cities. There is an efficient algorithm for solving this problem, called **Dijkstra's algorithm**. There is an explanation of Dijkstra's algorithm in the Week 1 folder on the module page on My Learning.

Task 1

A system of cities joined by roads is an example of a **graph**. In general a graph has **vertices** (e.g. cities) joined by **edges** (e.g. roads). Edges may have **weights** or **distances** attached to them. For this coursework we will assume that a road represents the same distance in either direction.

Write Python code to define classes to represent an Edge and a Graph. Your code should represent vertices as strings.

An Edge should have two vertices as its endpoints, and an optional real-number distance.

A Graph should have a list of Edges. Defining an edge whose two ends are the same doesn't make sense for this problem, so your Graph class should forbid it (i.e. your

code should raise a helpful Exception if the caller attempts to define an edge whose ends are the same). Similarly defining two edges between the same two vertices doesn't make sense, so your Graph class should forbid it.

The Graph class should provide methods to

- List the vertices of the graph.
- Determine whether two vertices are **adjacent**, i.e. whether there is an edge between them.
- Find the **neighbours** of a given vertex, i.e. the other vertices adjacent to it.

Task 2

Write a function that takes a list of triples (A, B, W) , where A and B are strings representing the ends of an edge and W is a real number representing its distance, and returns a Graph with Edges represented by the triples.

Task 3

JSON is a simple data format that can be used to save Python objects and read them back again. Documentation on JSON is available at <https://www.json.org/json-en.html>, and documentation on a Python library to handle JSON files is available at <https://docs.python.org/3/library/json.html>.

Write a function that reads a list of triples from a JSON file and returns the Graph defined by those triples.

Task 4

Write a method for your Graph class to implement Dijkstra's algorithm using your classes from Task 1. The method should take two strings representing vertices, and return the shortest path between those vertices and the length of that path.

Task 5

The *Assessment and Feedback* folder on the module's page on My Learning contains 5 JSON files representing graphs as in Task 3. Use your code to find the shortest route from vertex A to vertex Z in each of these graphs.

Submission

Your submission should consist of Zip archive as a single *.zip file containing:

1. One or more files of Python 3 code as *.py files.

Your code should run directly from the command line in a standard Python environment.

Your code may import one of your files into another, but this should work when all the files are stored in the same folder.

2. JSON data as a *.json file defining a graph of your choice as in Task 3. Your graph should have at least 8 vertices, distances on each edge, and include a path from vertex A to vertex Z.

3. A PDF report of up to 500 words as a *.pdf file. This should include

- Your forename, surname and student number.
- Instructions on how to use your code to find the shortest path in a graph defined using a JSON file as in Task 3. This should include details of any external libraries that should be installed in the Python environment.
- The results of running your code on the 5 graphs provided and on your graph defined in item 2 above.
- Details of two situations in which your code raises an Exception, and the reasons it does so.
- An idea for improving or extending your code, given more time or more advanced skills.

You should submit your Zip archive using the *Coursework submission* link in the *Assessment and Feedback* folder on the module's page on My Learning.

This coursework is marked anonymously, so please don't put anything in any file or in any filename that might identify you.

Mark scheme

Marks will be awarded for

- Success: Does your code run successfully?
- Correctness: Does your code produce the correct results?
Does your code deal successfully with error cases?
- Ease of use: How easy is it for the user to use your code?
- Flexibility: If an element of a task were changed, how easy would it to revise your code accordingly?
- Readability: How easy is it for another programmer to read and understand your code?
Does your code have comments to explain how it works?
Are the variable and function names helpful?
Is your code indented and spaced helpfully?
- Report: Does your report provide clear instructions on how to use your code, and details of any libraries required?
Does your report explain clearly the results of running your code on the various graphs? Bonus marks will be awarded for an explanation of how you know that your code worked correctly for your graph defined in item 2 of the Zip archive, i.e. how you know that the path that your code found is indeed the shortest path.
Does your report explain clearly when and why your code raises two Exceptions?
Does your report explain clearly an interesting idea for improving your code?